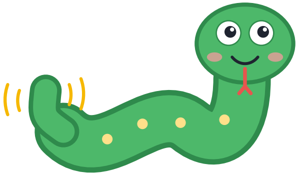
Introduction to Programming and Algorithms in Python

Lecture 9

Part 4: Data Science and AI

Objects, NumPy & Pandas

Build your own type, then use Python’s power tools: arrays, photos as numbers, tables



Dror Jacoby

Where we are

Part 1:
Intro and Python Basics

- **L1** Hello, Python!
- **L2** Variables & Types
- **L3** Input & Output

Part 2:
Decisions and Loops

- **L4** Making Decisions
- **L5** Lists & Sets
- **L6** Loops

Part 3:
Functions and Dictionaries

- **L7** Functions
- **L8** Dicts & Debugging

Part 4:
Data Science and AI

- **L9** Objects, NumPy & Pandas
- **L10** Plots & ML

You are here

Looking back: lectures 7 and 8

✓ Lecture 7: Functions

- `def` names a list of steps, a call runs them
- Parameters bring values in, `return` sends an answer out
- `print` shows, `return` hands back. Defaults: “the usual”
- Variables made inside a function stay inside

✓ Lecture 8: Dicts & Debugging

- A dictionary maps keys to values: look up, `in`, `get`, loop
- Tables are lists of rows. Median = middle of the sorted list
- Read errors bottom-up, print values, test AI code
- `try` / `except ValueError`: catch instead of crashing

Warm-up: lectures 7 and 8 in one program

```
def count_votes(votes):
    tally = {}
    for v in votes:
        tally[v] = tally.get(v, 0) + 1
    return tally
votes = ["tea", "tea", "cola", "tea"]
result = count_votes(votes)
print(result)
print(result["tea"])
print(result.get("beer", 0))
```

Warm-up: lectures 7 and 8 in one program

```python
def count_votes(votes):
    tally = {}
    for v in votes:
        tally[v] = tally.get(v, 0) + 1
    return tally
votes = ["tea", "tea", "cola", "tea"]
result = count_votes(votes)
print(result)
print(result["tea"])
print(result.get("beer", 0))
```

OUTPUT

```
{'tea': 3, 'cola': 1}
3
0
```

Quick check: predict the output

```python
def to_number(text, default=0):
    try:
        return int(text)
    except ValueError:
        return default

print(to_number("42"))
print(to_number("4.5"))
print(to_number("hi", -1))
```

Quick check: predict the output

```python
def to_number(text, default=0):
    try:
        return int(text)
    except ValueError:
        return default

print(to_number("42"))
print(to_number("4.5"))
print(to_number("hi", -1))
```

OUTPUT

```
42
0
-1
```

Quick check: predict the output

```python
def to_number(text, default=0):
    try:
        return int(text)
    except ValueError:
        return default

print(to_number("42"))
print(to_number("4.5"))
print(to_number("hi", -1))
```

OUTPUT

```
42
0
-1
```

- `int("42")` works, so it returns 42
- `int("4.5")` raises `ValueError`: the default 0 comes back
- −1 replaces the default for `"hi"`

Your coffee habit, in numbers

- 40 coffee orders this month
- Each order has a drink, a size, a price: one *thing*

Your coffee habit, in numbers

- 40 coffee orders this month
- Each order has a drink, a size, a price: one *thing*
- Add VAT to every price?

Your coffee habit, in numbers

- 40 coffee orders this month
- Each order has a drink, a size, a price: one *thing*
- Add VAT to every price?
- Average price? Most popular drink?

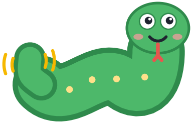
Your coffee habit, in numbers

- 40 coffee orders this month
- Each order has a drink, a size, a price: one *thing*
- Add VAT to every price?
- Average price? Most popular drink?
- And wait: is this cat **also** just numbers?

Today's plan

◎ **By the end of today you can...**

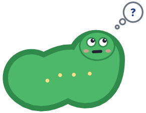
- Build your own type of object with a `class`
- Do math on many numbers at once with NumPy
- Edit a photo by changing its numbers
- Load a table with Pandas and ask it questions



Today's outline

1 **Objects and classes**

Every value is an object; build your own: `class`, `__init__`, `self`, methods



2 **Libraries & NumPy**

`import`, arrays, math on everything at once, filters, statistics, 2-D arrays

3 **An image is just numbers**

A photo as an array of pixels; filters are just math

4 **Pandas: tables in Python**

DataFrames, reading CSV files, columns, filtering rows, counting

Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Classes keep data and actions together. Every library is built from them
- Real data has thousands of rows. Loops are slow, NumPy is fast
- Pandas is Excel with superpowers, and it never crashes on big files

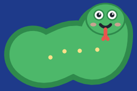
Where you will see it

- Research data in every faculty
- Finance, marketing, sports statistics
- Phone camera filters

Topic 1

Objects and classes

Lecture 9 • Objects, NumPy & Pandas



TODAY

1. **Objects and classes**
2. Libraries & NumPy
3. An image is just numbers
4. Pandas: tables in Python

Every value is an object

```
print(type(5))
print(type("hi"))
print(type([1, 2]))
print(type({"tea": 8}))
```

Every value is an object

```python
print(type(5))
print(type("hi"))
print(type([1, 2]))
print(type({"tea": 8}))
```

OUTPUT

```
<class 'int'>
<class 'str'>
<class 'list'>
<class 'dict'>
```

- Remember `type()` from Lecture 2?

Every value is an object

```python
print(type(5))
print(type("hi"))
print(type([1, 2]))
print(type({"tea": 8}))
```

OUTPUT

```
<class 'int'>
<class 'str'>
<class 'list'>
<class 'dict'>
```

- Remember `type()` from Lecture 2?
- A **class** = a type, a kind of thing
- An **object** = one value of that type

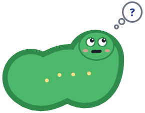
Every value is an object

```python
print(type(5))
print(type("hi"))
print(type([1, 2]))
print(type({"tea": 8}))
```

OUTPUT

```
<class 'int'>
<class 'str'>
<class 'list'>
<class 'dict'>
```

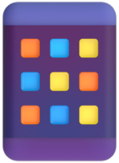
- Remember `type()` from Lecture 2?
- A **class** = a type, a kind of thing
- An **object** = one value of that type
- 5 and 7 are two objects of class `int`



Things have data and actions

Your phone is an object

Data: contacts, battery level
Actions: call, take a photo

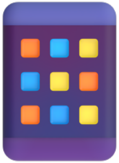


Things have data and actions

Your phone is an object

Data: contacts, battery level
Actions: call, take a photo

- Data that belongs to an object = an **attribute**
- An action that belongs to an object = a **method**

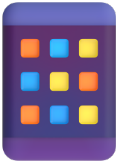


Things have data and actions

Your phone is an object

Data: contacts, battery level
Actions: call, take a photo

- Data that belongs to an object = an **attribute**
- An action that belongs to an object = a **method**
- In Python: `thing.data` and `thing.action()`



You have used methods all along

```python
cart = ["milk", "eggs"]
cart.append("pita")
print(cart)

name = "dana"
print(name.upper())

menu = {"tea": 8}
print(menu.get("cake", 0))
```

You have used methods all along

```
cart = ["milk", "eggs"]
cart.append("pita")
print(cart)

name = "dana"
print(name.upper())

menu = {"tea": 8}
print(menu.get("cake", 0))
```

OUTPUT

```
['milk', 'eggs',
'pita']
DANA
0
```

You have used methods all along

```python
cart = ["milk", "eggs"]
cart.append("pita")
print(cart)

name = "dana"
print(name.upper())

menu = {"tea": 8}
print(menu.get("cake", 0))
```

OUTPUT

```
['milk', 'eggs',
'pita']
DANA
0
```

Each class comes with its own methods: a list can `append`, a string can `upper`.

Today: build your own class.

Why build your own?

Two points on a map, with what we know:

```
x1 = 0
y1 = 0
x2 = 3
y2 = 4

home = {"x": 0, "y": 0}
work = {"x": 3, "y": 4}
```

Why build your own?

Two points on a map, with what we know:

```python
x1 = 0
y1 = 0
x2 = 3
y2 = 4

home = {"x": 0, "y": 0}
work = {"x": 3, "y": 4}
```

- Four loose variables: easy to mix up

Why build your own?

Two points on a map, with what we know:

```
x1 = 0
y1 = 0
x2 = 3
y2 = 4

home = {"x": 0, "y": 0}
work = {"x": 3, "y": 4}
```

- Four loose variables: easy to mix up
- Dictionaries: better, but `home["x"]` is long, and the actions live somewhere else

Why build your own?

Two points on a map, with what we know:

```
x1 = 0
y1 = 0
x2 = 3
y2 = 4

home = {"x": 0, "y": 0}
work = {"x": 3, "y": 4}
```

- Four loose variables: easy to mix up
- Dictionaries: better, but `home["x"]` is long, and the actions live somewhere else
- A **class**: data *and* actions in one box

A class is a **cookie cutter**. Each object is one cookie: same shape, its own toppings.

🍪

`class Point:` your own type

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)
print(p.x, p.y)
print(type(p))
```

`class Point`: your own type

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)
print(p.x, p.y)
print(type(p))
```

OUTPUT

```
3 4
<class '__main__.Point'>
```

`class Point`: your own type

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)
print(p.x, p.y)
print(type(p))
```

OUTPUT

```
3 4
<class '__main__.Point'>
```

- `class Point:` the cookie cutter. Class names start with a capital letter

`class Point:` your own type

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)
print(p.x, p.y)
print(type(p))
```

OUTPUT

```
3 4
<class '__main__.Point'>
```

- `class Point:` the cookie cutter. Class names start with a capital letter
- `Point(3, 4)` makes a new object. Python runs `__init__` (“init” = start)

`class Point`: your own type

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)
print(p.x, p.y)
print(type(p))
```

OUTPUT

```
3 4
<class '__main__.Point'>
```

- **`class`** `Point:` the cookie cutter. Class names start with a capital letter
- `Point(3, 4)` makes a new object. Python runs `__init__` (“init” = start)
- `self` = “this object”. `self.x = x` stores `x` *inside* it

Add a method: a function inside the class

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_to(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx ** 2 + dy ** 2) ** 0.5

home = Point(0, 0)
work = Point(3, 4)
print(home.distance_to(work))
```

Add a method: a function inside the class

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_to(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx ** 2 + dy ** 2) ** 0.5

home = Point(0, 0)
work = Point(3, 4)
print(home.distance_to(work))
```

OUTPUT

```
5.0
```

Add a method: a function inside the class

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_to(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx ** 2 + dy ** 2) ** 0.5

home = Point(0, 0)
work = Point(3, 4)
print(home.distance_to(work))
```

OUTPUT

```
5.0
```

- A method is a **`def`** (Lecture 7) inside a class. First parameter: `self`

Add a method: a function inside the class

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_to(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx ** 2 + dy ** 2) ** 0.5

home = Point(0, 0)
work = Point(3, 4)
print(home.distance_to(work))
```

OUTPUT

```
5.0
```

- A method is a **`def`** (Lecture 7) inside a class. First parameter: `self`
- In `home.distance_to(work)`, `self` is home

Biology: a dish of bacteria

```python
class Culture:
    def __init__(self, name, cells):
        self.name = name
        self.cells = cells

    def grow(self, hours):
        for hour in range(hours):
            self.cells = self.cells * 2

    def report(self):
        print(f"{self.name}: {self.cells} cells")
```

```python
dish = Culture("E. coli", 100)
dish.report()
dish.grow(3)
dish.report()
```

Biology: a dish of bacteria

```python
class Culture:
    def __init__(self, name, cells):
        self.name = name
        self.cells = cells

    def grow(self, hours):
        for hour in range(hours):
            self.cells = self.cells * 2

    def report(self):
        print(f"{self.name}: {self.cells} cells")
```

```python
dish = Culture("E. coli", 100)
dish.report()
dish.grow(3)
dish.report()
```

OUTPUT

```
E. coli: 100 cells
E. coli: 800 cells
```

Biology: a dish of bacteria

```python
class Culture:
    def __init__(self, name, cells):
        self.name = name
        self.cells = cells

    def grow(self, hours):
        for hour in range(hours):
            self.cells = self.cells * 2

    def report(self):
        print(f"{self.name}: {self.cells} cells")
```

```python
dish = Culture("E. coli", 100)
dish.report()
dish.grow(3)
dish.report()
```

OUTPUT

```
E. coli: 100 cells
E. coli: 800 cells
```

A method can **change** the data: `grow` doubles `self.cells`.

Toy model: cells double every hour. 🔬

Money: a bank account in ILS

```python
class BankAccount:
    def __init__(self, owner):
        self.owner = owner
        self.balance = 0

    def deposit(self, amount):
        self.balance = self.balance + amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money!")
        else:
            self.balance = self.balance - amount
```

```python
account = BankAccount("Noa")
account.deposit(500)
account.withdraw(800)
account.withdraw(200)
print(account.owner,
      account.balance, "ILS")
```

Money: a bank account in ILS

```python
class BankAccount:
    def __init__(self, owner):
        self.owner = owner
        self.balance = 0

    def deposit(self, amount):
        self.balance = self.balance + amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money!")
        else:
            self.balance = self.balance - amount
```

```python
account = BankAccount("Noa")
account.deposit(500)
account.withdraw(800)
account.withdraw(200)
print(account.owner,
      account.balance, "ILS")
```

OUTPUT

```
Not enough money!
Noa 300 ILS
```

Money: a bank account in ILS

```python
class BankAccount:
    def __init__(self, owner):
        self.owner = owner
        self.balance = 0

    def deposit(self, amount):
        self.balance = self.balance + amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money!")
        else:
            self.balance = self.balance - amount
```

```python
account = BankAccount("Noa")
account.deposit(500)
account.withdraw(800)
account.withdraw(200)
print(account.owner,
      account.balance, "ILS")
```

OUTPUT

```
Not enough money!
Noa 300 ILS
```

`balance` starts at 0: not every attribute is a parameter. The rule “no negative balance” lives **inside** the class.

Try it!

Try it in Colab `notebooks/L09/L09_00_objects_and_classes.ipynb`

① Make two points and print the distance between them

② Add a method `report()` to `BankAccount`

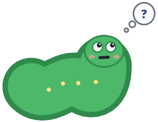
③ Write a class `Student`: a name, a list of grades, and a method `average()`

Predict the output: two dishes

? Predict the output!

```python
a = Culture("Yeast", 10)
b = Culture("Yeast", 10)
a.grow(2)
print(a.cells, b.cells)
```

The `Culture` class from two slides back.



Predict the output: two dishes

Predict the output!

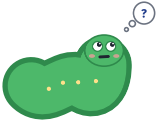
```
a = Culture("Yeast", 10)
b = Culture("Yeast", 10)
a.grow(2)
print(a.cells, b.cells)
```

The `Culture` class from two slides back.

Answer

```
40 10
```

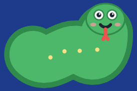
`a` grew twice: $10 \to 20 \to 40$.
`b` is a different object with its own `cells`. It did not change.



Topic 2

Libraries & NumPy

Lecture 9 • Objects, NumPy & Pandas



TODAY

1 Objects and classes

2 Libraries & NumPy

3 An image is just numbers

4 Pandas: tables in Python

Libraries: an app store for code

- Someone already wrote the hard parts
- A **library** = ready-made tools
- You **import** it, then use it
- Colab already has them installed

🔢
NumPy
numbers

📒
Pandas
tables

📈
Matplotlib
plots (L10)

🤖
scikit-learn
AI (L10)

import: open the toolbox

`L09_01_numpy_basics.ipynb`

```
import numpy as np

print(np.sqrt(16))
print(np.pi)
```

import: open the toolbox

`L09_01_numpy_basics.ipynb`

```python
import numpy as np

print(np.sqrt(16))
print(np.pi)
```

OUTPUT

```
4.0
3.141592653589793
```

import: open the toolbox

L09_01_numpy_basics.ipynb

```python
import numpy as np

print(np.sqrt(16))
print(np.pi)
```

OUTPUT

```
4.0
3.141592653589793
```

- `as np` gives NumPy a short nickname
- `np.sqrt` means: the `sqrt` tool *from* NumPy

Meet the array

```
import numpy as np

prices = np.array([12, 15, 9, 20])
print(prices)
print(len(prices))
print(prices[0])
```

An **array** is a list built for math.

No commas when printed. Same indexes as lists.

Meet the array

```
import numpy as np

prices = np.array([12, 15, 9, 20])
print(prices)
print(len(prices))
print(prices[0])
```

OUTPUT

```
[12 15  9 20]
4
12
```

An **array** is a list built for math.

No commas when printed. Same indexes as lists.

List vs array: same look, different powers

```
import numpy as np

prices_list = [12, 15, 9, 20]
prices = np.array([12, 15, 9, 20])
print(prices_list * 2)
print(prices * 2)
```

List vs array: same look, different powers

```python
import numpy as np

prices_list = [12, 15, 9, 20]
prices = np.array([12, 15, 9, 20])
print(prices_list * 2)
print(prices * 2)
```

OUTPUT

```
[12, 15, 9, 20, 12, 15, 9, 20]
[24 30 18 40]
```

List vs array: same look, different powers

```python
import numpy as np

prices_list = [12, 15, 9, 20]
prices = np.array([12, 15, 9, 20])
print(prices_list * 2)
print(prices * 2)
```

OUTPUT

```
[12, 15, 9, 20, 12, 15, 9, 20]
[24 30 18 40]
```

List * 2: **repeat** the list. Array * 2: **double every number**.

Add 18% VAT to every price

The L06 way: a loop

```
prices_list = [12, 15, 9, 20]
with_vat = []
for price in prices_list:
    with_vat.append(price * 1.18)
```

The NumPy way: one line

```
import numpy as np

prices = np.array([12, 15, 9, 20])
with_vat = prices * 1.18
print(with_vat)
```

Add 18% VAT to every price

The L06 way: a loop

```python
prices_list = [12, 15, 9, 20]
with_vat = []
for price in prices_list:
    with_vat.append(price * 1.18)
```

The NumPy way: one line

```python
import numpy as np

prices = np.array([12, 15, 9, 20])
with_vat = prices * 1.18
print(with_vat)
```

OUTPUT

```
[14.16 17.7  10.62 23.6 ]
```

Add 18% VAT to every price

The L06 way: a loop

```python
prices_list = [12, 15, 9, 20]
with_vat = []
for price in prices_list:
    with_vat.append(price * 1.18)
```

The NumPy way: one line

```python
import numpy as np

prices = np.array([12, 15, 9, 20])
with_vat = prices * 1.18
print(with_vat)
```

OUTPUT

```
[14.16 17.7  10.62 23.6 ]
```

Math on the whole array = math on every item.

Two arrays, item by item

```
import numpy as np

homework = np.array([90, 80, 100, 70])
exam = np.array([70, 85, 60, 95])

final = 0.3 * homework + 0.7 * exam
print(final)
```

Four students’ final grades.

Item 1 with item 1, item 2 with item 2…

Two arrays, item by item

```python
import numpy as np

homework = np.array([90, 80, 100, 70])
exam = np.array([70, 85, 60, 95])

final = 0.3 * homework + 0.7 * exam
print(final)
```

OUTPUT

```
[76.  83.5 72.  87.5]
```

Four students’ final grades.

Item 1 with item 1, item 2 with item 2…

Ask the array a question

```
import numpy as np

prices = np.array([12, 15, 9, 20])
print(prices > 12)
print(prices[prices > 12])
```

Ask the array a question

```python
import numpy as np

prices = np.array([12, 15, 9, 20])
print(prices > 12)
print(prices[prices > 12])
```

OUTPUT

```
[False  True False  True]
[15 20]
```

Ask the array a question

```python
import numpy as np

prices = np.array([12, 15, 9, 20])
print(prices > 12)
print(prices[prices > 12])
```

OUTPUT

```
[False  True False  True]
[15 20]
```

- `prices > 12` asks *every* price: True or False?

Ask the array a question

```python
import numpy as np

prices = np.array([12, 15, 9, 20])
print(prices > 12)
print(prices[prices > 12])
```

OUTPUT

```
[False  True False  True]
[15 20]
```

- `prices > 12` asks *every* price: True or False?
- `prices[...]` keeps only the `True` ones

Statistics in one word each

```
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(np.mean(grades))
print(np.median(grades))
print(np.percentile(grades, 90))
print(np.max(grades))
```

Statistics in one word each

```python
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(np.mean(grades))
print(np.median(grades))
print(np.percentile(grades, 90))
print(np.max(grades))
```

OUTPUT

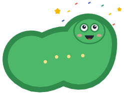
```
81.0
88.0
96.8
100
```

Statistics in one word each

```python
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(np.mean(grades))
print(np.median(grades))
print(np.percentile(grades, 90))
print(np.max(grades))
```

```
OUTPUT
81.0
88.0
96.8
100
```



Remember the median **by hand** in L08? Now it is one line.

`percentile(grades, 90)`: 90% of grades are below this.

A 2-D array is a table

	Test 1	Test 2	Test 3	Test 4
Dana	90	80	70	60
Omer	60	75	85	100
Noa	90	100	85	89

- Rows: students
- Columns: tests

(counting from 0, as always)

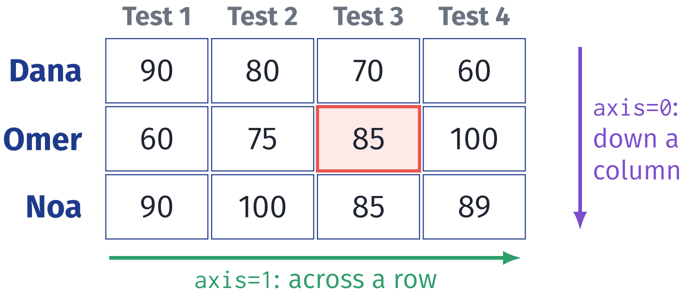
A 2-D array is a table

	Test 1	Test 2	Test 3	Test 4
Dana	90	80	70	60
Omer	60	75	85	100
Noa	90	100	85	89

- Rows: students
- Columns: tests
- Row 1, column 2 is **85**

(counting from 0, as always)

A 2-D array is a table



	Test 1	Test 2	Test 3	Test 4
Dana	90	80	70	60
Omer	60	75	85	100
Noa	90	100	85	89

`axis=0`: down a column

`axis=1`: across a row

- Rows: students
- Columns: tests
- Row 1, column 2 is **85**

(counting from 0, as always)

2-D arrays in code

```python
import numpy as np

grades = np.array([[90, 80, 70, 60],
                   [60, 75, 85, 100],
                   [90, 100, 85, 89]])
print(grades.shape)
print(grades[1, 2])
print(grades.max())
```

2-D arrays in code

```python
import numpy as np

grades = np.array([[90, 80, 70, 60],
                   [60, 75, 85, 100],
                   [90, 100, 85, 89]])
print(grades.shape)
print(grades[1, 2])
print(grades.max())
```

OUTPUT

```
(3, 4)
85
100
```

2-D arrays in code

```python
import numpy as np

grades = np.array([[90, 80, 70, 60],
                   [60, 75, 85, 100],
                   [90, 100, 85, 89]])
print(grades.shape)
print(grades[1, 2])
print(grades.max())
```

OUTPUT

```
(3, 4)
85
100
```

- shape: (rows, columns)
- `grades[row, col]`

Average per student, average per test

```python
import numpy as np

grades = np.array([[90, 80, 70, 60],
                   [60, 75, 85, 100],
                   [90, 100, 85, 89]])
print(grades.mean(axis=1))   # each student (row)
print(grades.mean(axis=0))   # each test (column)
```

Average per student, average per test

```python
import numpy as np

grades = np.array([[90, 80, 70, 60],
                   [60, 75, 85, 100],
                   [90, 100, 85, 89]])
print(grades.mean(axis=1))   # each student (row)
print(grades.mean(axis=0))   # each test (column)
```

OUTPUT

```
[75. 80. 91.]
[80. 85. 80. 83.]
```

Average per student, average per test

```python
import numpy as np

grades = np.array([[90, 80, 70, 60],
                   [60, 75, 85, 100],
                   [90, 100, 85, 89]])
print(grades.mean(axis=1))   # each student (row)
print(grades.mean(axis=0))   # each test (column)
```

OUTPUT

```
[75. 80. 91.]
[80. 85. 80. 83.]
```

Dana 75, Omer 80, Noa 91. Test 2 was the easiest (85).

An array is an object too

```
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(type(grades))
print(grades.shape)
print(grades.mean())
```

An array is an object too

```python
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(type(grades))
print(grades.shape)
print(grades.mean())
```

OUTPUT

```
<class 'numpy.ndarray'>
(5,)
81.0
```

An array is an object too

```python
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(type(grades))
print(grades.shape)
print(grades.mean())
```

OUTPUT

```
<class 'numpy.ndarray'>
(5,)
81.0
```

- Someone wrote `class ndarray`, just like our `Point`

An array is an object too

```python
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(type(grades))
print(grades.shape)
print(grades.mean())
```

OUTPUT

```
<class 'numpy.ndarray'>
(5,)
81.0
```

- Someone wrote `class ndarray`, just like our `Point`
- `grades.shape`: an **attribute** (data), no `()`
- `grades.mean()`: a **method** (action), with `()`

An array is an object too

```python
import numpy as np

grades = np.array([55, 70, 88, 92, 100])
print(type(grades))
print(grades.shape)
print(grades.mean())
```

OUTPUT

```
<class 'numpy.ndarray'>
(5,)
81.0
```

- Someone wrote `class ndarray`, just like our Point
- `grades.shape`: an **attribute** (data), no `()`
- `grades.mean()`: a **method** (action), with `()`
- Pandas tables and AI models (Lecture 10) are objects too

Try it!

Try it in Colab `notebooks/L09/L09_01_numpy_basics.ipynb`

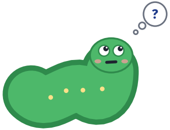
① Make an array of 5 coffee prices you paid this week
② Add 18% VAT to all of them in one line
③ Which prices are above 15? Keep only those
④ Print the mean and the median

Predict the output

Predict the output!

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a[a > 15] - 10)
```



Predict the output

? Predict the output!

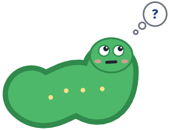
```python
import numpy as np

a = np.array([10, 20, 30, 40])
print(a[a > 15] - 10)
```

✓ Answer

```
[10 20 30]
```

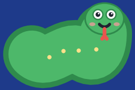
Keep 20, 30, 40. Then subtract 10 from each.



Topic 3

An image is just numbers

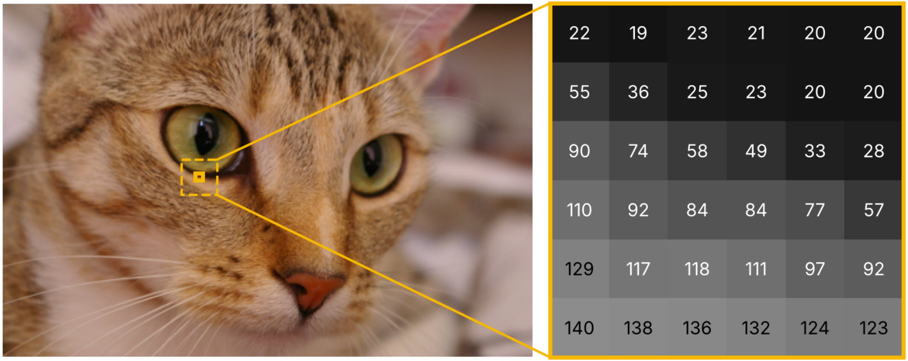
Lecture 9 • Objects, NumPy & Pandas



TODAY

1. Objects and classes
2. Libraries & NumPy
3. **An image is just numbers**
4. Pandas: tables in Python

Zoom in on a photo

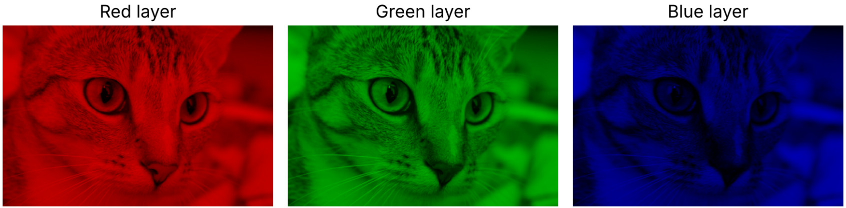


22	19	23	21	20	20
55	36	25	23	20	20
90	74	58	49	33	28
110	92	84	84	77	57
129	117	118	111	97	92
140	138	136	132	124	123

Zoom in enough: tiny squares called **pixels**. Each one is a **number**.

0 = black, 255 = white, in between = gray

Color = three numbers per pixel


Red layer
Green layer
Blue layer

- Every pixel: **R**ed, **G**reen, **B**lue
- Each from 0 (none) to 255 (full)

🟥 `[255 0 0]` red

🟨 `[255 255 0]` yellow

⬜ `[255 255 255]` white

🟫 `[167 119 73]` the cat's fur

Load a real photo: Chelsea the cat

`L09_02_cat_filters.ipynb`

```
from skimage import data

cat = data.chelsea()
print(cat.shape)
print(cat[150, 170])
```

`skimage` = scikit-image, a library of photo tools with sample images

Load a real photo: Chelsea the cat

`L09_02_cat_filters.ipynb`

```python
from skimage import data

cat = data.chelsea()
print(cat.shape)
print(cat[150, 170])
```

OUTPUT

```
(300, 451, 3)
[167 119  73]
```

`skimage` = scikit-image, a library of photo tools with sample images

Load a real photo: Chelsea the cat

L09_02_cat_filters.ipynb

```python
from skimage import data

cat = data.chelsea()
print(cat.shape)
print(cat[150, 170])
```

OUTPUT

```
(300, 451, 3)
[167 119  73]
```

- 300 rows of pixels
- 451 columns
- 3 colors per pixel

`skimage` = scikit-image, a library of photo tools with sample images

Load a real photo: Chelsea the cat

`L09_02_cat_filters.ipynb`

```python
from skimage import data

cat = data.chelsea()
print(cat.shape)
print(cat[150, 170])
```

OUTPUT

```
(300, 451, 3)
[167 119  73]
```

- 300 rows of pixels
- 451 columns
- 3 colors per pixel
- One pixel: lots of red, less green, little blue

`skimage` = scikit-image, a library of photo tools with sample images

Show the array as a picture

```python
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
plt.imshow(cat)
plt.show()
```

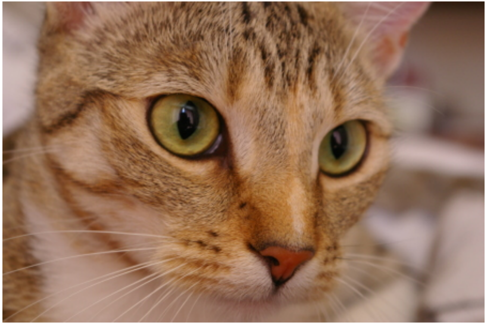
`plt.imshow`: “show this array as an image”.

Matplotlib gets a full tour next week.

Show the array as a picture

```
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
plt.imshow(cat)
plt.show()
```



`plt.imshow`: “show this array as an image”.

Matplotlib gets a full tour next week.

Filter 1: invert the colors

```python
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
inverted = 255 - cat
plt.imshow(inverted)
plt.show()
```

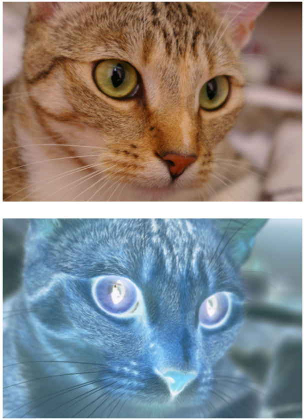
`255 - cat` changes **every** number:
dark becomes light, light becomes dark.

Filter 1: invert the colors

```
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
inverted = 255 - cat
plt.imshow(inverted)
plt.show()
```

`255 - cat` changes **every** number:
dark becomes light, light becomes dark.



Filter 2: crop with slicing

```
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
eyes = cat[70:210, 110:350]
print(eyes.shape)
plt.imshow(eyes)
plt.show()
```

Rows 70 to 210, columns 110 to 350.

Slicing from L05, now in 2-D!

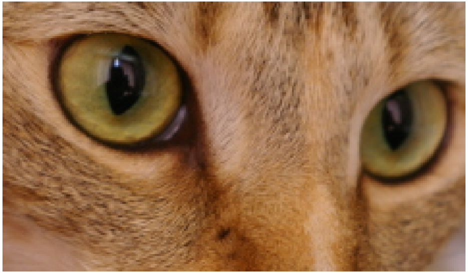
Filter 2: crop with slicing

```python
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
eyes = cat[70:210, 110:350]
print(eyes.shape)
plt.imshow(eyes)
plt.show()
```

OUTPUT

```
(140, 240, 3)
```



Rows 70 to 210, columns 110 to 350.

Slicing from L05, now in 2-D!

Filter 3: black and white

```
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
gray = cat.mean(axis=2)
print(gray.shape)
plt.imshow(gray, cmap="gray")
plt.show()
```

Average of R, G, B for every pixel.

`axis=2`: the color direction.

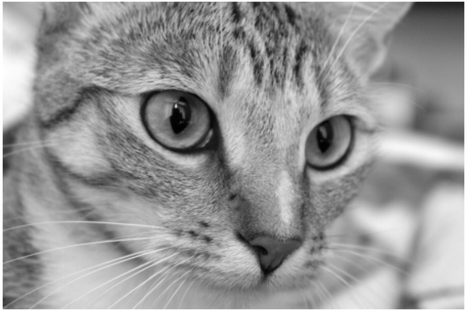
Filter 3: black and white

```python
from skimage import data
import matplotlib.pyplot as plt

cat = data.chelsea()
gray = cat.mean(axis=2)
print(gray.shape)
plt.imshow(gray, cmap="gray")
plt.show()
```

OUTPUT

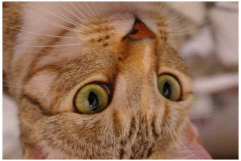
```
(300, 451)
```



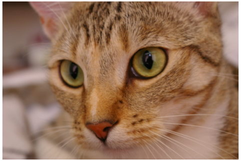
Average of R, G, B for every pixel.

`axis=2`: the color direction.

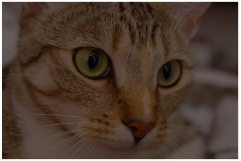
Photo filters are just math


`cat[::-1]`

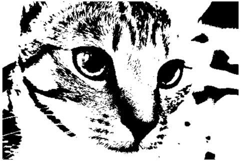
upside down


`cat[:, ::-1]`

mirror


`cat // 2`

darker


`gray > 100`

pure black & white

- Step −1 reverses, just like in L05
- Phone filters, medical scans, self-driving cars: all pixel math

Try it!

Try it in Colab

notebooks/L09/L09_02_cat_filters.ipynb

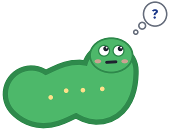
① Crop the cat's nose: rows 220 to 300, columns 230 to 330
② Make it upside down **and** inverted
③ Change `gray > 100` to 50, then 150. What happens?
④ Bonus: upload your own photo and try the same filters

Predict the output

? Predict the output!

```
from skimage import data

cat = data.chelsea()
small = cat[0:100, 0:50]
print(small.shape)
```



Predict the output

? Predict the output!

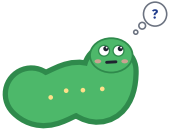
```
from skimage import data

cat = data.chelsea()
small = cat[0:100, 0:50]
print(small.shape)
```

✓ Answer

```
(100, 50, 3)
```

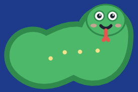
100 rows, 50 columns, still 3 colors.



Topic 4

Pandas: tables in Python

Lecture 9 • Objects, NumPy & Pandas



TODAY

1. Objects and classes
2. Libraries & NumPy
3. An image is just numbers
4. **Pandas: tables in Python**

Pandas = a spreadsheet inside Python

- A **DataFrame** is a table
- Each **column** has a name
- Each **row** is one record (one order)
- Like Excel, but you ask with code

	drink	price
0	Latte	16
1	Tea	8
2	Mocha	18

A DataFrame from a dictionary

`L09_03_coffee_pandas.ipynb`

```
import pandas as pd

menu = {"drink": ["Latte", "Tea", "Mocha"],
        "price": [16, 8, 18]}
df = pd.DataFrame(menu)
print(df)
```

A DataFrame from a dictionary

L09_03_coffee_pandas.ipynb

```python
import pandas as pd

menu = {"drink": ["Latte", "Tea", "Mocha"],
        "price": [16, 8, 18]}
df = pd.DataFrame(menu)
print(df)
```

OUTPUT

```
   drink  price
0  Latte     16
1    Tea      8
2  Mocha     18
```

A DataFrame from a dictionary

L09_03_coffee_pandas.ipynb

```python
import pandas as pd

menu = {"drink": ["Latte", "Tea", "Mocha"],
        "price": [16, 8, 18]}
df = pd.DataFrame(menu)
print(df)
```

OUTPUT

```
   drink  price
0  Latte     16
1    Tea      8
2  Mocha     18
```

- Dictionary keys become **column names**
- The numbers on the left are row numbers (the **index**)

Real data lives in CSV files

`coffee_orders.csv`

```
day,drink,size,price,rating
Sun,Cappuccino,S,13,5
Sun,Espresso,S,9,5
Sun,Latte,L,18,3
...
```

- CSV = **C**omma-**S**eparated **V**alues
- A plain text file; commas split the columns
- Excel and Google Sheets can save as CSV

Today's data

40 orders from a campus café. **Made-up data**, created for this course.

Read the CSV, peek at the top

```
import pandas as pd
df = pd.read_csv("coffee_orders.csv")
print(df.head())
```

Read the CSV, peek at the top

```python
import pandas as pd
df = pd.read_csv("coffee_orders.csv")
print(df.head())
```

OUTPUT

```
   day       drink size  price  rating
0  Sun  Cappuccino    S     13       5
1  Sun    Espresso    S      9       5
2  Sun       Latte    L     18       3
3  Sun         Tea    S      8       2
4  Sun       Latte    M     16       3
```

Read the CSV, peek at the top

```python
import pandas as pd
df = pd.read_csv("coffee_orders.csv")
print(df.head())
```

- head(): the first 5 rows
- In Colab: upload the file first (folder icon, left side)

OUTPUT

```
   day       drink size  price  rating
0  Sun  Cappuccino    S     13       5
1  Sun    Espresso    S      9       5
2  Sun       Latte    L     18       3
3  Sun         Tea    S      8       2
4  Sun       Latte    M     16       3
```

One column = one array of numbers

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df["price"].sum())
print(df["price"].mean())
print(df["price"].median())
print(df["price"].quantile(0.9))
```

One column = one array of numbers

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df["price"].sum())
print(df["price"].mean())
print(df["price"].median())
print(df["price"].quantile(0.9))
```

OUTPUT

```
538
13.45
14.0
18.1
```

One column = one array of numbers

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df["price"].sum())
print(df["price"].mean())
print(df["price"].median())
print(df["price"].quantile(0.9))
```

OUTPUT

```
538
13.45
14.0
18.1
```

Total spent: 538 ILS.
Average order: 13.45 ILS.

`df["price"]` picks one column, like a dictionary key.

Filter rows: only the lattes

```
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
lattes = df[df["drink"] == "Latte"]
print(len(lattes))
print(round(lattes["price"].mean(), 2))
```

Same trick as
`prices[prices > 12]`:
a True/False question for every row.

Filter rows: only the lattes

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
lattes = df[df["drink"] == "Latte"]
print(len(lattes))
print(round(lattes["price"].mean(), 2))
```

OUTPUT

```
14
16.14
```

Same trick as `prices[prices > 12]`: a True/False question for every row.

Filter rows: the expensive orders

```
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
expensive = df[df["price"] >= 19]
print(expensive)
```

Filter rows: the expensive orders

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
expensive = df[df["price"] >= 19]
print(expensive)
```

OUTPUT

```
    day        drink size  price  rating
24  Wed  Iced coffee    L     19       4
30  Wed  Iced coffee    L     19       3
34  Thu  Iced coffee    L     19       5
38  Thu  Iced coffee    L     19       5
```

Filter rows: the expensive orders

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
expensive = df[df["price"] >= 19]
print(expensive)
```

All four: large iced coffee.

Rows keep their numbers.

OUTPUT

```
    day        drink size  price  rating
24  Wed  Iced coffee    L     19       4
30  Wed  Iced coffee    L     19       3
34  Thu  Iced coffee    L     19       5
38  Thu  Iced coffee    L     19       5
```

Which drink is the most popular?

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df["drink"].value_counts())
```

Counts each value, biggest first.

Like the word counter from L08.

Which drink is the most popular?

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df["drink"].value_counts())
```

OUTPUT

```
drink
Latte          14
Espresso       10
Tea             6
Cappuccino      5
Iced coffee     5
Name: count, dtype: int64
```

Counts each value, biggest first.
Like the word counter from L08.

describe(): the whole summary at once

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df.describe())
```

describe(): the whole summary at once

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df.describe())
```

OUTPUT

```
           price    rating
count  40.000000  40.00000
mean   13.450000   3.82500
std     3.908898   0.98417
min     8.000000   2.00000
25%     9.000000   3.00000
50%    14.000000   4.00000
75%    17.000000   5.00000
max    19.000000   5.00000
```

describe(): the whole summary at once

```python
import pandas as pd

df = pd.read_csv("coffee_orders.csv")
print(df.describe())
```

- std: how spread out the values are
- 25%, 50%, 75%: quantiles

OUTPUT

```
           price    rating
count  40.000000  40.00000
mean   13.450000   3.82500
std     3.908898   0.98417
min     8.000000   2.00000
25%     9.000000   3.00000
50%    14.000000   4.00000
75%    17.000000   5.00000
max    19.000000   5.00000
```

Try it!

Try it in Colab

`notebooks/L09/L09_03_coffee_pandas.ipynb`

① Load `coffee_orders.csv` and print `head()`

② How many orders were on Wednesday?

③ What is the average `rating` of Tea?

④ Which day had the most orders? (try `value_counts`)

Predict the output

Predict the output!

```
import pandas as pd
menu = {"drink": ["Latte", "Tea", "Mocha"],
        "price": [16, 8, 18]}
df = pd.DataFrame(menu)
cheap = df[df["price"] < 17]
print(len(cheap))
print(cheap["price"].mean())
```

Predict the output

? Predict the output!

```
import pandas as pd
menu = {"drink": ["Latte", "Tea", "Mocha"],
        "price": [16, 8, 18]}
df = pd.DataFrame(menu)
cheap = df[df["price"] < 17]
print(len(cheap))
print(cheap["price"].mean())
```

✓ Answer

```
2
12.0
```

Latte (16) and Tea (8) are under 17. Their average is 12.

Bug Zoo: the forgotten `self`

BUG ZOO

```
class Point:
    def __init__(self, x, y):
        x = x
        y = y

p = Point(3, 4)
print(p.x)
```

ERROR

```
AttributeError: 'Point' object
has no attribute 'x'
```

Fix: `self.x = x` stores it *in* the object

```
class Culture:
    def __init__(self, name, cells):
        self.name = name
        self.cells = cells

    def report():
        print(self.cells)

Culture("E. coli", 100).report()
```

ERROR

```
TypeError: Culture.report() takes 0
positional arguments but 1 was given
```

Fix: `def report(self):`

Bug Zoo: the forgotten import

BUG ZOO

```
prices = np.array([12, 15, 9, 20])
```

ERROR

```
NameError: name 'np' is not defined
```

Fix: run `import numpy as np` first (top cell!)

```
prices = [12, 15, 9, 20]
with_vat = prices * 1.18
```

ERROR

```
TypeError: can't multiply
sequence by non-int of
type 'float'
```

Fix: a list can’t do math on items. Use `np.array(prices)`.

Bug Zoo: file and column names

BUG ZOO

```
df = pd.read_csv("coffee_order.csv")
```

ERROR

```
FileNotFoundError: [Errno 2] No such
file or directory: 'coffee_order.csv'
```

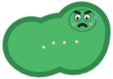
Fix: check the spelling, and upload the file to Colab.

```
print(df["Price"].mean())
```

ERROR

```
KeyError: 'Price'
```

Fix: column names are exact: `"price"`. Check `df.columns`.



The intern looked for a file and a column that don’t exist.

AI Corner: ask about your table

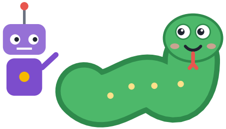
AI CORNER

A good prompt

“My Pandas table `df` has columns day, drink, size, price. How do I find the average price of each drink? Explain every line simply.”

Double-check

Filter one drink and average it yourself. Same number?



Share column names, never private data.

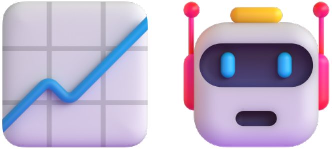
Recap

✓ **Today you learned**

- A `class` is a cookie cutter: `__init__` stores data in `self`, methods are actions
- NumPy arrays do math on every item at once
- A photo is an array of numbers: edit numbers, edit photo
- Pandas reads CSV tables: filter rows, then `mean()`

Next time: pictures from data, and a taste of AI

- Turn numbers into plots with Matplotlib
- Teach the computer from examples (scikit-learn)
- Code like a pro: a real editor, a debugger, an AI assistant
- Look back at the whole course, and your final project



3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md